Department of Software Engineering
Lakehead University

ESOF 2570 – Object Oriented Graphical User Interface

Term Project: Cannon Game App

Due: Tue, Mar 31, 2026, 2:30 pm

Project Overview

Design and develop a *Cannon Game App* as illustrated in Figure 1.

- All required sound files are included in the project folder.
- The project should demonstrate correct use of object-oriented principles and JavaFX graphical user interface features.

This game requires the use of two-dimensional analytic geometry and basic trigonometry to calculate the cannon's firing angle. All required sound files are provided in the project folder, although you may also use or create your own.

The Cannon game app challenges the player to destroy targets before a 10-second time limit expires. The game features a cannon, a cannonball, nine targets, and a blocker that protects the targets. The blocker cannot be destroyed. The blocker and each target move vertically at different speeds, reversing direction when they reach the top or bottom of the **Canvas**. The player aims and fires the cannon using the mouse: the cannon's barrel rotates to point toward the click location, and the cannonball is then fired in a straight line in that direction.

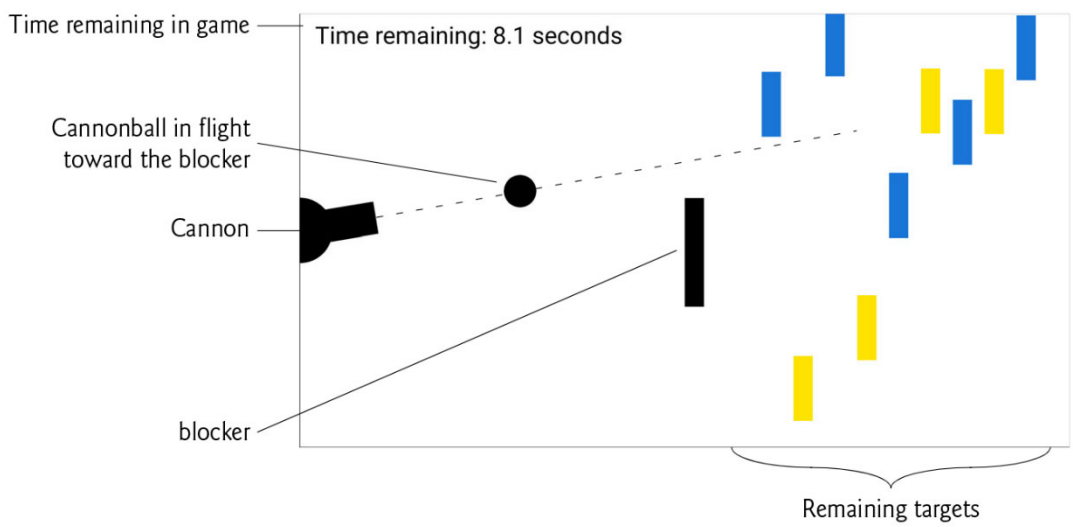

Figure 1. Cannon game app

The player wins the game by destroying all nine targets before the time expires; if the timer reaches zero, the player loses. Destroying a target increases the remaining time by a three-second bonus, while hitting the blocker results in a three-second time penalty. When the game ends, the app displays an **Alert** dialog (package: **javafx.scene.control**) indicating whether the player won or lost, along with the total number of shots fired and the elapsed time.

When the cannon is fired, the game plays a *firing sound* (**cannon_fire.wav**), and a cannonball begins moving from the cannon's barrel toward the blocker and targets. When a cannonball hits a target, a *glass-breaking*

sound (**target_hit.wav**) plays and that target disappears. When the cannonball hits the blocker, a *hit sound* (**blocker_hit.wav**) plays, and the cannonball bounces back toward the cannon.

Use a **Canvas** to implement the game's graphics. This app should perform its animations manually by updating the game elements using **AnimationTimer**, as demonstrated in Section 22.9. Consider defining separate classes to represent the various game elements; each class may define its own **AnimationTimer** to control the motion of that element.

For simplicity, perform collision detection using the cannonball's bounding box—that is, the square area in which the cannonball is drawn—rather than its circular shape. A collision is considered to have occurred if the cannonball's bounding box intersects with the bounds of the blocker or any target.

[Many of today's games are implemented with game-development frameworks that provide sophisticated "pixel-perfect" collision-detection capabilities. Some frameworks include physics engines that can simulate real-world attributes like mass, gravity, friction, speed, acceleration and more. There are various free and fee-based frameworks available for developing from the simplest 2D games to the most complex 3D console-style games (such as games for Sony's PlayStation® and Microsoft's Xbox®)].

Submission Policy

- Late submissions are accepted up to two days after the deadline.
- Penalty: 25% of the total marks per day late.

Marking Rubrics

Aspect/Level	1 – Failed (1–35%)	2 – Unsatisfactory (36–59%)	3 – Meets with Restrictions (60–79%)	4 – Meets (80–89%)	5 – Exceeds (90–100%)
Program correctness	Most requirements missing or incorrect.	Some requirements implemented, several errors; may not run.	Most requirements implemented; runs with minor issues	All requirements implemented; runs correctly and fully functional.	Exceeds expectations; exceptional implementation and accuracy.
Readability & Coding	Code unorganized, hard to read, violates standards.	Poorly organized, lengthy, unclear naming, violates standards.	Fairly readable and organized; some efficiency maintained.	Readable, organized, efficient; consistent naming and standards.	Exceptionally readable, efficient, well-organized; follows best practices.
Program Design & Style	No evidence of design or modularization.	Few appropriate structures; poor naming, indentation, modularization.	Some structures well-designed; partial modularization.	Good design, proper structures, modularization, naming, syntax.	Excellent design, fully appropriate structures, modularization, and conventions.
User Interface	No usable interface.	Limited UI; incomplete interaction, does not meet specs.	Basic UI; meets minimal specifications.	Good UI; meets specifications and user-friendly.	Excellent UI; intuitive, natural, fully meets specs.
Sound Integration	No sound used.	Sounds partially implemented or inconsistent.	Basic use of sounds; functional but minimal.	All sounds correctly implemented; enhances experience.	Creative sound use; enhances gameplay and immersion.
Creativity & Extras	No extra features.	Minimal enhancements; little added value.	Some extra features; minor added value.	Good additional features; improves user experience.	Highly creative features; significantly improves gameplay or appeal.